No.	Basic PHP Questions
	What is the PHP?
	Which is the latest version of PHP?
	What Type of Framework in Php?
	What Type of CMS(Content Management System) in Php?
	Full Form of LAMP?
	Full Form of WAMP?
	Full Form of XAMPP?
	PHP Life Cycle

What is the PHP?

- PHP is an open source server side scripting language used to develop dynamic websites. PHP stands for Hypertext Preprocessor , also stood for Personal Home Page. It was created by Rasmus lerdorf in 1995 . It is free software released under the PHP license.
- PHP is an acronym for "PHP: Hypertext Pre-processor" And Old name of PHP personal home page.
- Rasmus Lerdorf is known as the father of PHP. 1994
- PHP is a server side scripting language/s/w/tool commonly used for web applications. And PHP has many framework and CMS for creating a website.
- PHP is a widely-used, open source scripting language. And server side scripting language.
- PHP it is used to manage dynamic content, databases, session tracking, even build entire e-commerce sites.

Latest version of PHP?

The latest stable version of PHP is 8.2 released on _____.

Framework in Php?

Cakephp, Laravel, Codeigniter, Yii 2, Zend Framework, Phalcon, Slim, FuelPhp, Phpixie, etc

CMS(Content Management System) in Php?

Wordpress, Joomla, Magento, Drupal, etc

Full Form of LAMP?

Linux Apache MySql and Php.

Full Form of WAMP?

Windows Apache MySql And Php.

Full Form of XAMPP?

```
X-OS, Apache Mysql Php Perl
X: Any of the different operating system(Windows,Linux,Mac OS X), to be read as
“cross”, meaning cross-platform.
Apache(HTTP Server)
Mysql(Database)
PHP
Perl
```

PHP Life Cycle?

- PHP life cycle is the process in which a client request reaches the web server, the web server passes the PHP request to the PHP engine, PHP code is executed, required database or API operations are performed, a response is generated, and finally that response is sent back to the browser.

Steps

1. **User Request:-** Browser server ko request bhejta hai.
2. **Web Server:-** Apache/Nginx request receive karta hai aur identify karta hai ki ye PHP file hai.
3. **PHP Engine:-** PHP engine PHP code ko execute karta hai.
4. **Database/API interaction:-** Agar code mein database query hai: to application database se data fetch karegi.
5. **Response Generate:-** PHP execution ke baad response generate hota hai:
6. **Response Browser ko:-** Web server generated response browser ko bhej deta hai, aur browser page display karta hai.

```
Browser
   ↓
HTTP Request
   ↓
Web Server (Apache/Nginx)
   ↓
PHP Engine / PHP-FPM
   ↓
PHP Code Execute
   ↓
Database / APIs / Files
   ↓
HTML/JSON Response
   ↓
Web Server
   ↓
Browser
```

SOLID Principles

- SOLID object-oriented programming ke 5 important design principles hain. Inka purpose code ko clean, maintainable, scalable aur loosely coupled banana hai.
    - **S — Single Responsibility:** A class should have only one responsibility.
    - **O — Open/Closed:** Code should be open for extension but closed for modification.
    - **L — Liskov Substitution:** A child class should be able to replace its parent class without breaking the code.
    - **I — Interface Segregation:** A class should not be forced to implement unnecessary methods.
    - **D — Dependency Inversion:** Classes should depend on abstractions, not concrete classes.

1. **S — Single Responsibility Principle (SRP):-**

- A class should have only one responsibility.(Matlab ek class ko sirf ek responsibility handle karni chahiye.)

```
# ❌ Bad:
class User
{
    public function createUser() {}
    public function sendEmail() {}
    public function generateReport() {}
}

# ✅ Better:
class UserService
{
    public function createUser() {}
}

class EmailService
{
    public function sendEmail() {}
}

class ReportService
{
    public function generateReport() {}
}
```

2. **O — Open/Closed Principle (OCP)**

- Classes should be open for extension but closed for modification.
- Existing code ko baar-baar modify karne ke bajay new functionality extend karni chahiye.

```
interface Payment
{
    public function pay();
}

class StripePayment implements Payment
{
    public function pay()
    {
        echo "Stripe payment";
    }
}

class PaypalPayment implements Payment
{
    public function pay()
    {
        echo "Paypal payment";
    }
}
```

- Kal agar Razorpay add karna ho:

```
class RazorpayPayment implements Payment
{
    public function pay()
    {
        echo "Razorpay payment";
    }
}
```

- Existing classes ko modify karne ki zarurat nahi.

3. **L — Liskov Substitution Principle (LSP)**

- Child class should be replaceable with its parent class without breaking the application.
- Agar kisi parent class ki jagah uski child class ko use karein, to application ka behavior nahi badalna chahiye aur code break nahi hona chahiye.

```
class Bird
{
    public function eat()
    {
        echo "Eating";
    }
}

class Sparrow extends Bird
{
    public function fly()
    {
        echo "Flying";
    }
}
```

- Yahan Sparrow, Bird ka child hai. Jahan Bird object use ho sakta hai, wahan Sparrow bhi use ho sakta hai.

```
# WRONG Example
class Bird
{
    public function fly()
    {
        echo "Flying";
    }
}

class Penguin extends Bird
{
    public function fly()
    {
        throw new Exception("Penguin can't fly");
    }
}
```

- Problem:
    - Parent Bird bol raha hai ki sab birds fly kar sakte hain.
    - Penguin fly nahi kar sakta.
    - Isliye Penguin ko Bird ki jagah use karne par code break ho sakta hai.
- Correct code

```
interface Flyable
{
    public function fly();
}

class Sparrow implements Flyable
{
    public function fly() {}
}

class Penguin
{
    public function eat() {}
}
```

4. **I — Interface Segregation Principle (ISP)**

- A class should not be forced to implement methods that it does not need.

```
❌ Bad:
interface Worker
{
    public function work();
    public function eat();
    public function sleep();
}
```

- Agar Robot ko implement karna hai, to robot ko eat() aur sleep() ki zarurat nahi.

```
✅ Better:
interface Workable
{
    public function work();
}

interface Eatable
{
    public function eat();
}

interface Sleepable
{
    public function sleep();
}
```

- Ab Robot sirf:

```
class Robot implements Workable
{
    public function work() {}
}
```

5. D — Dependency Inversion Principle (DIP)

- High-level modules should depend on abstractions, not concrete implementations.

❌ Bad:

```
class OrderService
{
    private StripePayment $payment;

    public function __construct()
    {
        $this->payment = new StripePayment();
    }
}
# OrderService directly StripePayment par dependent hai.
```

✅ Better:

```
interface Payment
{
    public function pay();
}

class OrderService
{
    public function __construct(
        private Payment $payment
    ) {}

    public function order()
    {
        $this->payment->pay();
    }
}
# Ab hum easily Stripe, PayPal, Razorpay kuch bhi inject kar sakte hain.
```